Master 2 CIM - DA - DT 2026/2027
ICOM - Université de Lyon 2

Module Champ d'Application (Angular JS)

Travaux Pratiques 4

Animations, Routing

Objectif: L'objectif de cette séance de TP est de comprendre l'intégration d'animations dans *Angular* et nous introduirons le *Routing* comme extension.

Partie I: Génération de composants et styles SCSS

🎯 Apprendre à animer les composants avec *Angular* et CSS

Les animations bien conçues rendent une application plus agréable et intuitive. Elles ne sont pas seulement esthétiques : elles adoucissent les transitions, aident l'utilisateur à percevoir les réactions de l'application et guident son attention de manière fluide dans les différentes étapes.

Angular fournit deux éléments `animate.enter` et `animate.leave` pour animer les éléments de votre application. Ces deux éléments :

- appliquent les classes CSS enter et leave aux moments appropriés ;
  (ou appelle les fonctions pour appliquer les animations).
- `animate.enter` et `animate.leave` ne sont pas des directives.
- inclus dans API spéciale prise en charge directement par le compilateur.

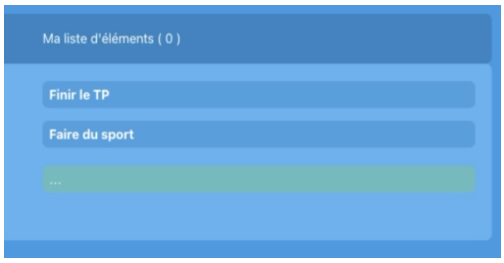
Précision : `animate.enter` s'applique à l'entrée dans le DOM, tandis que `animate.leave` intervient lors de la suppression de l'élément.

1.1 Pour animer un élément, il faut donc d'abord rajouter une classe CSS associée à une animation. Nous détaillerons dans la suite de ce module les mécanismes d'animation CSS. Pour la suite vous pourrez utiliser la base de l'animation suivante que vous devez recopier dans le fichier *home.scss*.

```
51  .animation-ajout {
52      animation: slide-fade 1s;
53    }
54    @keyframes slide-fade {
55      from {
56        opacity: 0;
57        transform: translateY(20px);
58      }
59      to {
60        opacity: 1;
61        transform: translateY(0);
62      }
63    }
```

1.2 Dans le composant dont vous voulez qu'il arrive animé, il suffit de rajouter simplement le `animate.enter=""` où le nom de la classe d'animation CSS sera à insérer dans les guillemets (par exemple *fade-in*). Tester d'intégrer l'animation pour animer un objectif lorsqu'il est rajouté dans l'interface.

1.3 Editez l'animation CSS de façon à ajouter des couleurs dans l'animation et avoir quelque chose ressemblant à l'image suivante :



Partie II: *Routing*

Le *routing* est une étape importante pour pouvoir gérer la navigation entre les pages de votre application.

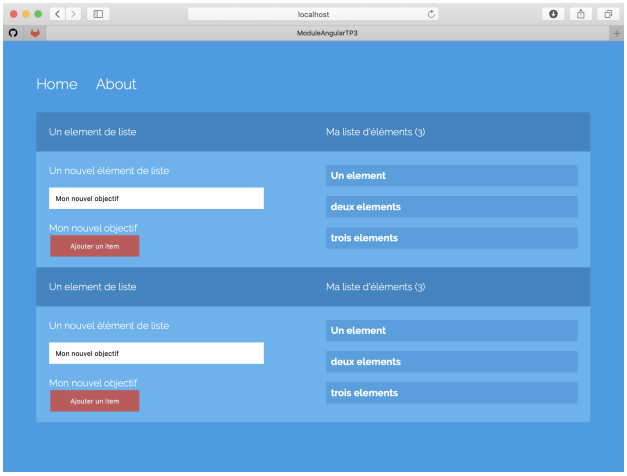
2.1 La première étape pour définir le *routing* consiste à modifier le fichier `app.routes.ts`. Ce fichier contient le tableau `routes`, qui permet d'associer un chemin (`path`) au composant (`component`) à afficher.

Dans ce fichier, importez les composants `Home` et `About` en précisant leur chemin relatif. Vous pouvez vous aider des fichiers `home.ts` et `about.ts` pour retrouver les chemins d'importation.

2.2 Ensuite la deuxième étape est de considérer le tableau `routes`. Ce tableau permet d'associer pour un chemin donné (`path`) à son composant (`component`). Le composant associé à la route sera affiché lorsque l'URL correspondra au chemin défini. Ajoutez les routes suivantes :

Chemin	Composant
`''`	`Home`
`'about'`	`About`

2.3 Comme vous pouvez le voir, la page affiche deux fois le composant `Home`. Corrigez le problème en utilisant uniquement `<router-outlet>` afin qu'un seul composant soit visible.



2.4 Dans le fichier `app.component.html`, mettez à jour la cible du lien de façon à pouvoir naviguer entre les deux pages *Home* et *About*.

Dans la suite, nous verrons comment il est aussi possible de transmettre des paramètres à une page de la même manière que nous pourrions le faire à travers le mécanisme classique d'une *Query-String*.

Partie III: *Routing et transmission de paramètres*

Afin d'étudier la transmission de paramètres, nous allons revoir le mécanisme de *routing* vu lors de la séance précédente.

3.1 Modifiez le tableau `routes` du fichier `src/app/app-routing.module.ts` de façon à avoir le comportement suivant :

- page par défaut : `http://localhost:4200/`
  → doit cibler le composant principal : *HomeComponent*
- page index : `http://localhost:4200/index.html`
  → doit cibler le composant principal : *HomeComponent*
- page about : `http://localhost:4200/about`
  → doit cibler sur le composant : *AboutComponent*

Vérifiez que les liens *Home* et *About* sont bien fonctionnels, tout comme l'accès à la page `index.html`

3.2 Afin de vérifier la différence entre les liens *routerLink* et *href*, rajoutez un nouveau lien *About2* dans le menu principal qui sera défini en utilisant un *href*. Vérifiez l'avantage que procure l'utilisation du lien *routerLink* en particulier lors du clic (navigation Angular côté client ou rechargement du document).

Dans les questions suivantes, l'objectif est de tester la fonction vue en cours, permettant de demander au routeur de charger une page (en *TypeScript*). En particulier, nous allons illustrer cette fonction en chargeant automatiquement la page *about* quelques secondes après que l'utilisateur ait rajouté un élément dans l'interface.

3.3 La première étape est de rajouter un paramètre de type `Router` dans le constructeur de la classe `HomeComponent` (fichier `src/app/home/home.component.ts`). Ajoutez le paramètre en utilisant le mot clé private suivi du nom de la variable `monRouteur` et vérifiez que vous pouvez bien utiliser la variable dans d'autres fonctions de votre classe.

3.4 Maintenant il ne reste plus qu'à déclencher un *timer* après que l'utilisateur ait appuyé sur le bouton `Ajouter un item` puis à appeler la fonction du routeur permettant le chargement d'une page (fonction `navigate`).
Indications : utiliser la fonction `setTimeout(()=> {}, 1000 );`

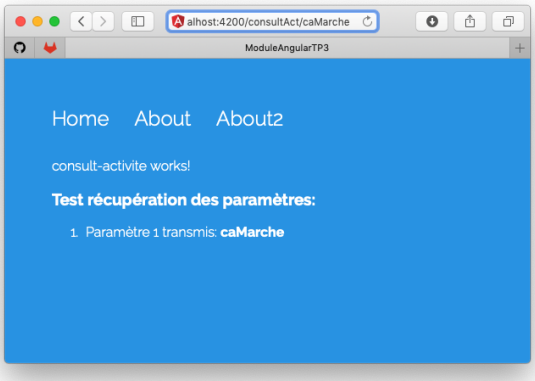
Pour terminer cette partie, nous allons illustrer la transmission de paramètres définie dans l'extension d'une URL.

3.5 Dans votre projet, depuis un terminal, **générez** un nouveau composant *ConsultActivite* et l'intégrer dans le routage de façon à ce qu'il soit accessible depuis l'url suivante : `http://localhost:4200/consultAct`. Vérifiez que la page est bien accessible.

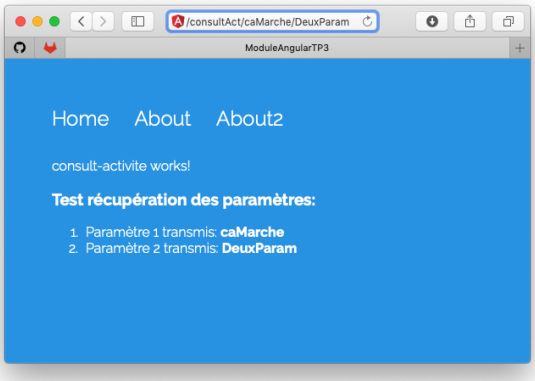
3.6 Dans le fichier de routage `app-routing.module.ts`, rajoutez une route pour accéder toujours à la page *consultAct* mais en pouvant rajouter un complément à l'adresse. Cette adresse contiendra un identifiant (voir cours : syntaxe :id) Par exemple, l'accès à la page `http://localhost:4200/consultAct/bonjour` doit pouvoir fonctionner.

3.7 Dans le constructeur de la classe `ConsultActiviteComponent`, rajoutez un paramètre de type `ActivatedRoute` et accédez à ses propriétés `snapshot.params` pour accéder au tableau associatif et récupérez la valeur associée au paramètre `id` que vous avez défini dans la question **2.6**. **Indication :** pour rajouter le paramètre dans le constructeur, n'oubliez d'importer la classe `ActivatedRoute` au début du fichier `consult-activite.component.ts`.

3.8 Exploitez la variable précédente de façon à mettre en évidence la transmission de paramètres. Vous pourrez par exemple obtenir l'affichage de l'image (a) :



(a)



(b)

3.9 Testez la transmission de plusieurs variables. Par exemple l'utilisation de l'url suivante donnera l'affichage de l'image (b) :

`http://localhost:4200/consultAct/caMarche/deuxParam`

3.10 En utilisant l'instruction `@if` à l'intérieur d'une balise `<li>`, faire en sorte que l'on obtienne l'affichage de l'image (a) si un seul paramètre n'est donné et l'affichage (b) si un deuxième paramètre est ajouté.